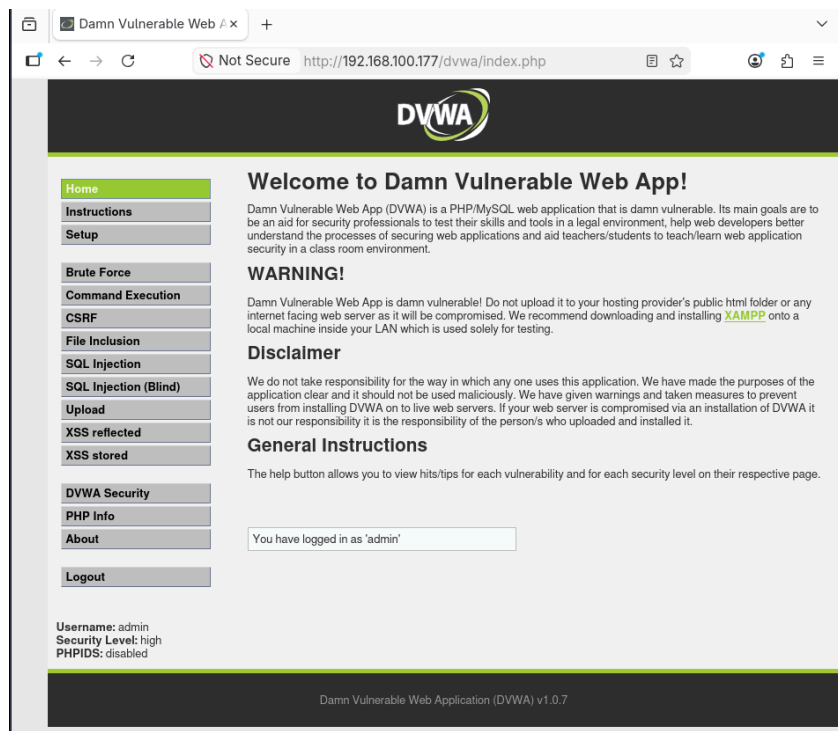




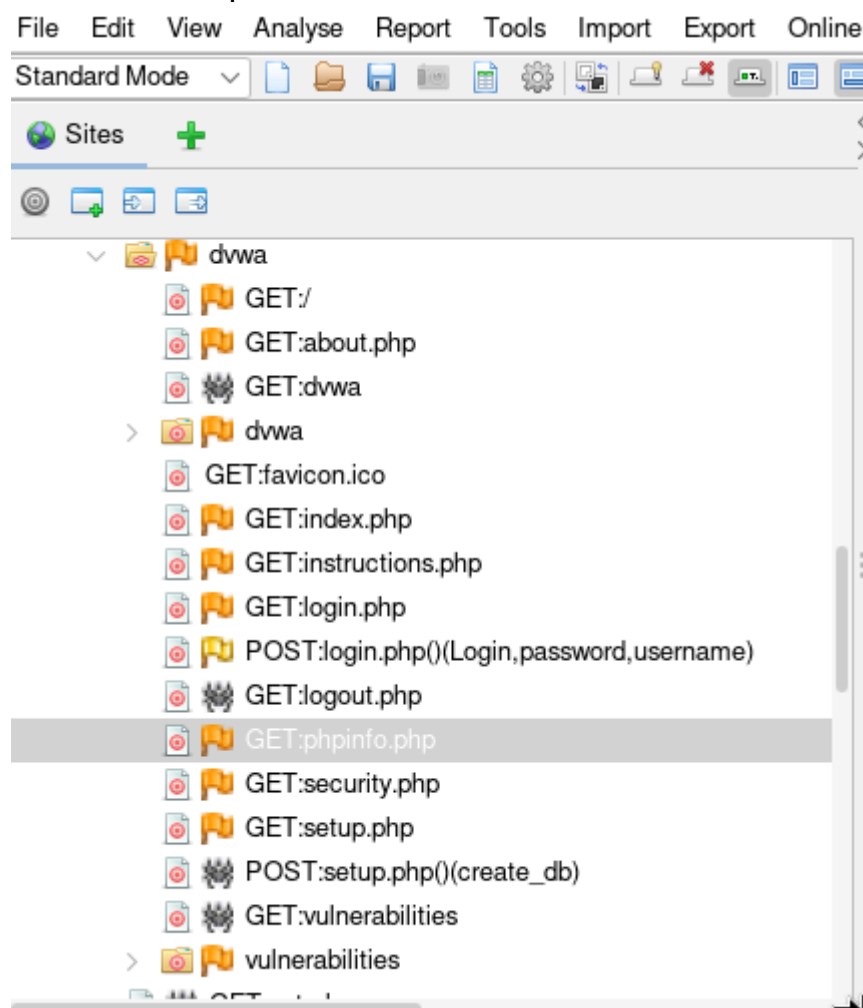
Student Name	Ayaan Raza	week	2
Track	Ethical Hacking	Instructor	Muhammad Saad

Task 1:

Installation Proof:

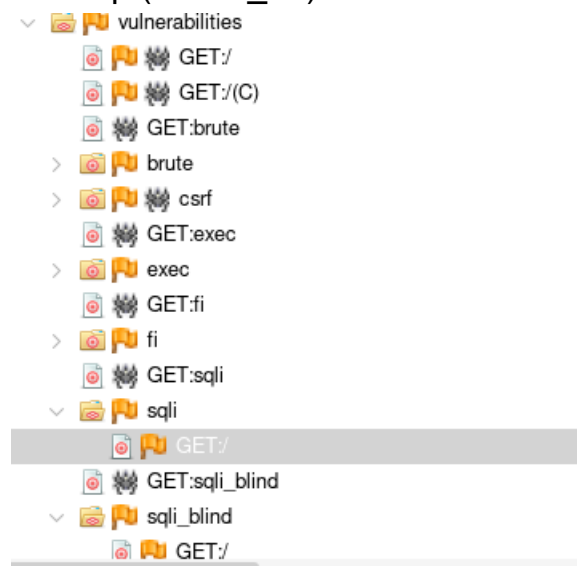


For all the endpoints



Now the endpoints which have input fields are all the ones with post
le in login(password,username)

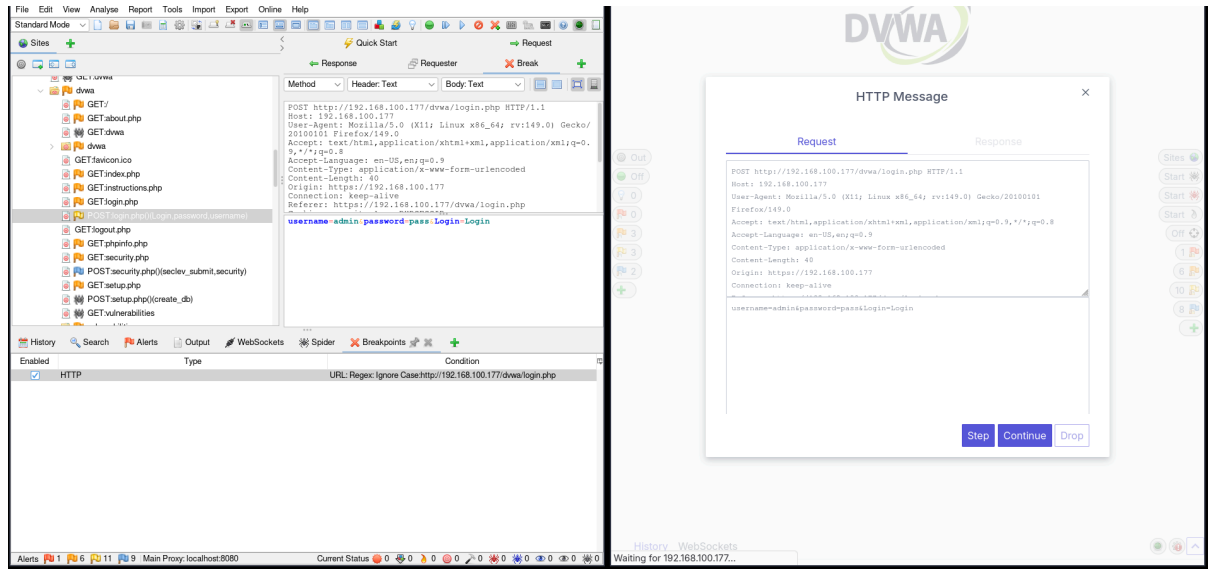
In setup (create_db)



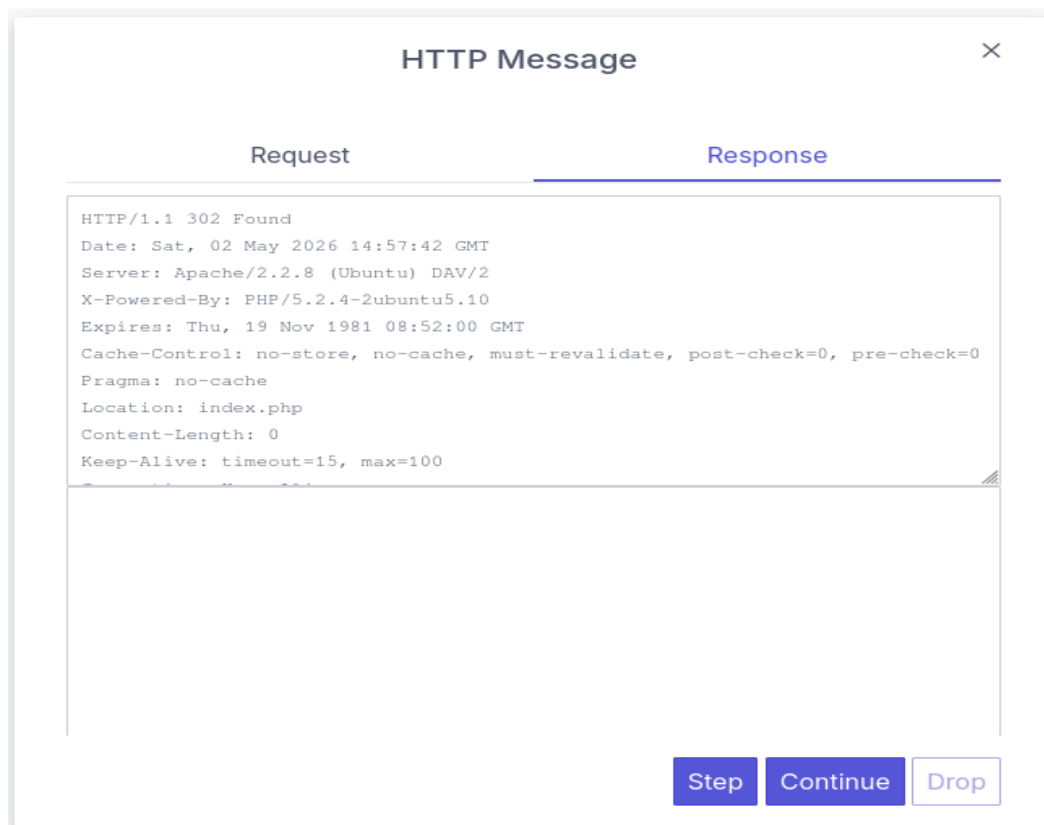
These are all the endpoints in DVWA

Task 2:

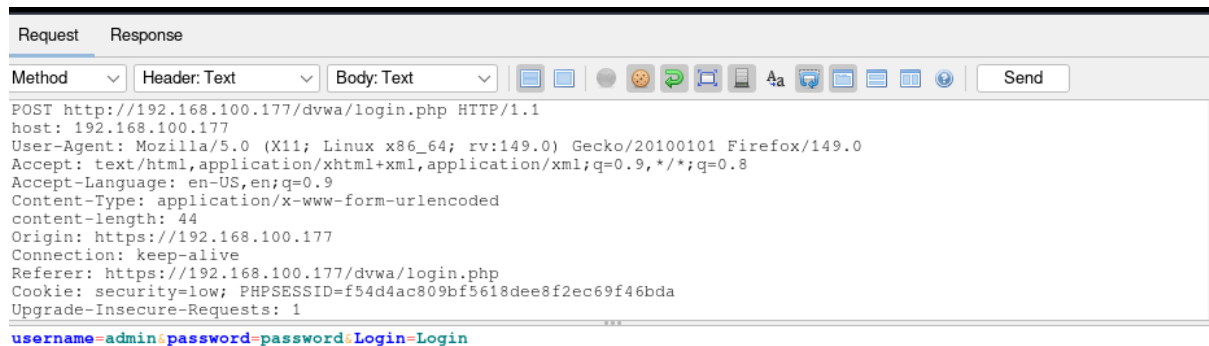
We first set ZAP/Burpsuit to intercept packets and then pressed the login button so that it can capture the request



Then pressed step so that it can be sent forward and we can get a response



And then we accessed the end point through the request editor



1

Theory:

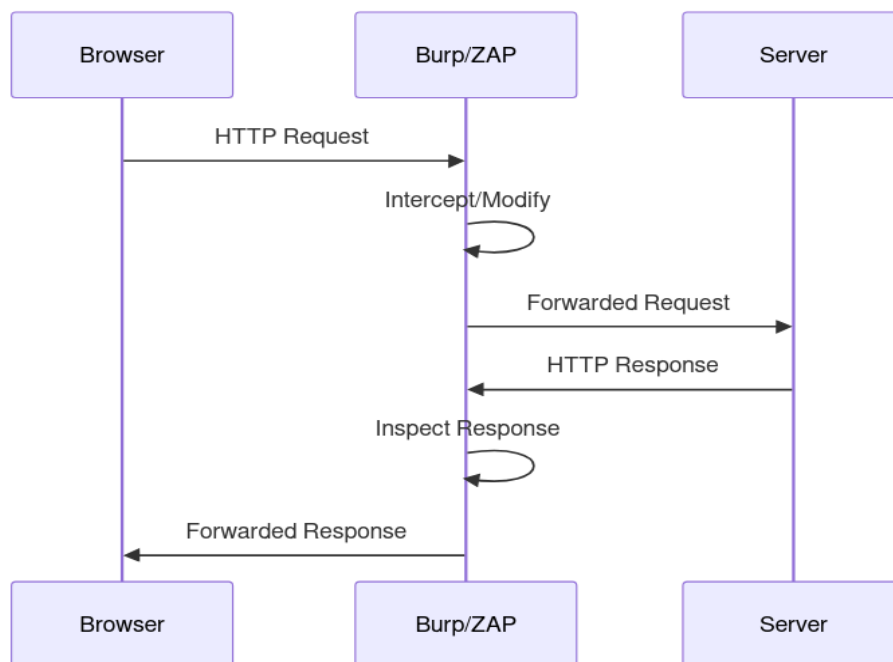
What is Burp Suite and how is it different from Nmap:

Burp Suite is a web application proxy, it sits between your browser and the server, intercepting and allowing modification of HTTP/S traffic. It operates at the **application layer** (Layer 7).

Nmap is a network scanner, it probes hosts for open ports and running services. It operates at the **network/transport layer** (Layers 3-4).

Burp asks: *what is this web app doing?* Nmap asks: *what ports/services are running on this machine?*

Burp/ZAP as an interceptor



Task 3

Requested Youtube

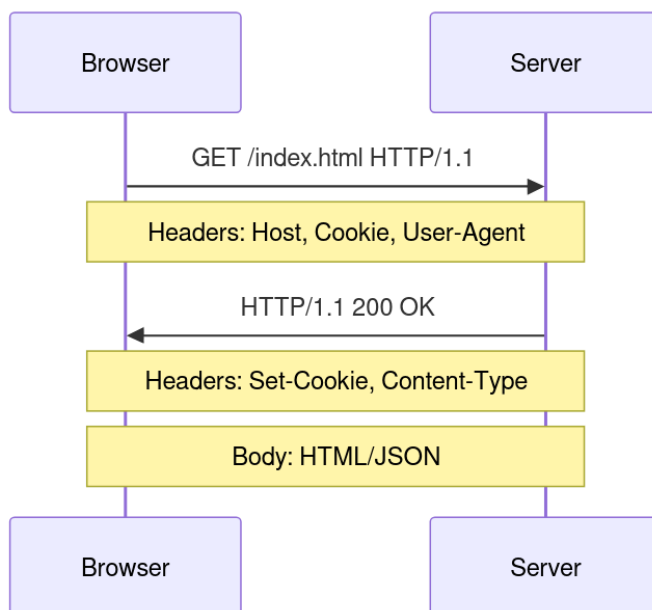
```
Header: Text  Body: Text
GET https://www.youtube.com/ HTTP/1.1
host: www.youtube.com
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Upgrade-Insecure-Requests: 1
Service-Worker-Navigation-Preload: true
Connection: keep-alive
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Priority: u=2
```

Response:

Header: Text Body: Text

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=utf-8
<-Content-Type-Options: nosniff
Content-Security-Policy: script-src 'unsafe-eval' 'self' 'unsafe-inline'
https://www.google.com https://apis.google.com https://ssl.gstatic.com
https://www.gstatic.com https://www.googletagmanager.com
https://www.google-analytics.com https://*.youtube.com https://*.google.com
https://*.gstatic.com https://youtube.com https://www.youtube.com https://google.com
https://*.doubleclick.net https://*.googleapis.com https://www.googleadservices.com
https://tpc.googlesyndication.com https://www.youtubekids.com
https://www.youtube-nocookie.com https://www.youtubeeducation.com
https://www.onepick-opensocial.googleusercontent.com;report-uri
https://csp.withgoogle.com/csp/youtube_main/allowlist
Content-Security-Policy: require-trusted-types-for 'script'
Content-Security-Policy: base-uri 'self';object-src 'none';script-src 'nonce-
300xs3ZnI490f4v0k9ic7a' 'unsafe-inline' 'strict-dynamic' https: http: 'unsafe-eval':
e\u003d\u0026html5_liveness_drift_proxima_override\u003d\u0026html5_log_audio_abr\u0026
quality_cap_for_inline_playback_ads\u003d\u0026read_ahead_model_name\u003d\u0026
delay_ping\u003dtrue\u0026deprecate_pair_servlet_enabled\u003dtrue\u0026desktop_spa
cs_with_debug_data\u003dtrue\u0026html5_log_vss_extra_lr_cparams_freq\u003d\u0026html
ove_chevron_from_ad_disclosure_banner_h5\u003dtrue\u0026remove_masthead_channel_banne
els\u003dtrue\u0026disable_child_node_auto_formatted_strings\u003dtrue\u0026disable_f
aplugged\u003dtrue\u0026html5_manifestless_vp9_otf\u003dtrue\u0026html5_max_buffer_he
ility_retriever_in_watch\u003dtrue\u0026return_drm_product_unknown_for_clear_playback
ole_lifa_for_supex_users\u003dtrue\u0026disable_log_to_visitor_layer\u003dtrue\u0026d
\u0026html5_max_byterate\u003d\u0026html5_max_discontinuity_rewrite_count\u003d\u0026
plate\u003dself_podding_interstitial_message\u0026self_podding_midroll_choice_string_
in_md5_module_for_music_web\u003dtrue\u0026disable_pacf_logging_for_memory_limited_t
r_track_secs\u003d0.0\u0026html5_max_headm_for_streaming_xhr\u003d\u0026html5_max_li
dding_midroll_choice\u0026send_config_hash_timer\u003d\u0026serve_adaptive_fmfs_for_
```

Theory



What is Request & Response in web context:

A request is a structured message sent by the browser to a server asking for a resource or action. A response is the server's structured reply containing a status, metadata, and optionally a body.

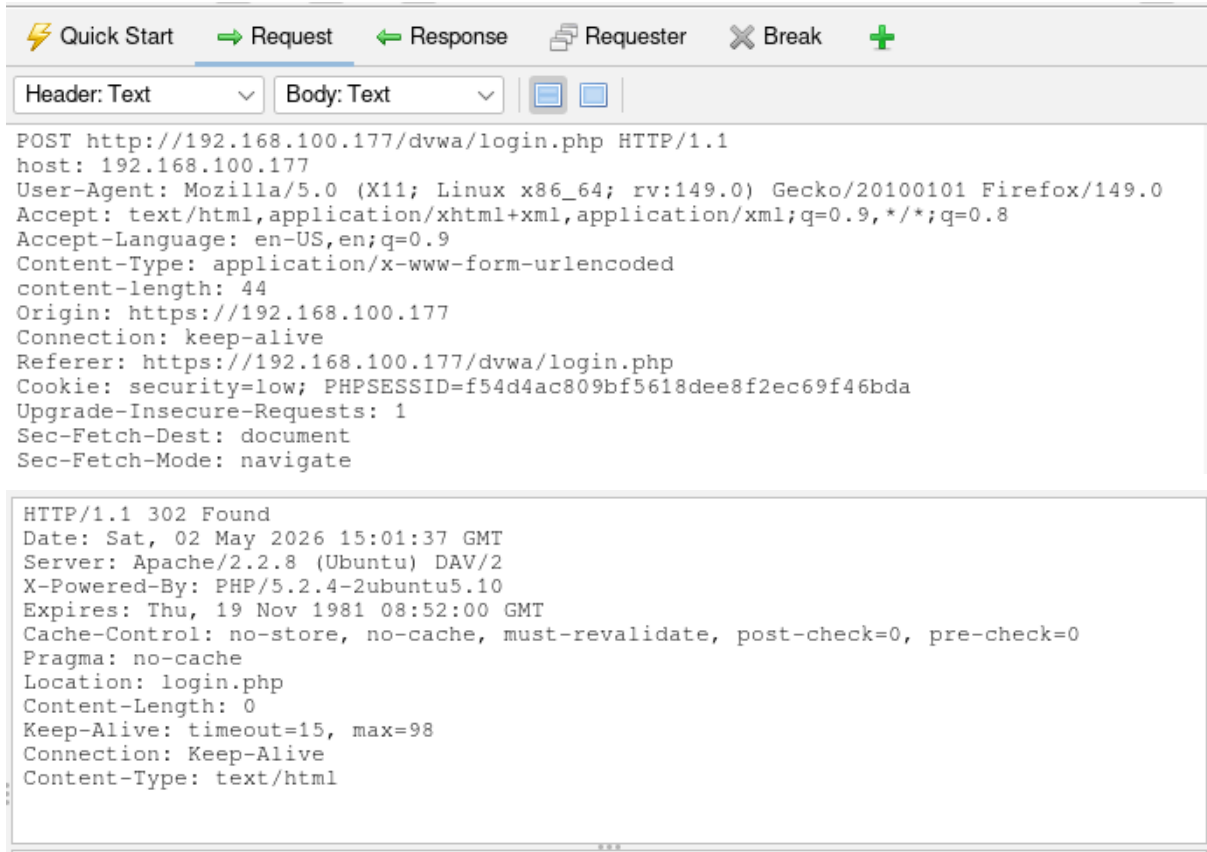
Sender/Receiver analogy:

The browser (sender) writes a letter specifying what it wants (verb, path, headers, body) and mails it. The server (receiver) reads it, processes it, and sends feedback (status code, headers, body) back.

Common HTTP response codes:

- 200 OK, success
- 301 / 302 Redirect
- 400 Bad request (client error)
- 401 Unauthorized (not authenticated)
- 403 Forbidden (not authorized)
- 404 Not found
- 500 Internal server error

Task 4



The screenshot shows a web browser's developer tools interface. The top bar has tabs for 'Quick Start', 'Request', 'Response', 'Requester', 'Break', and a plus sign. The 'Request' tab is selected. Below the tabs, there are two dropdown menus: 'Header: Text' and 'Body: Text'. The main area displays the details of an HTTP POST request to `http://192.168.100.177/dvwa/login.php`. The request headers include `host: 192.168.100.177`, `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0`, `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`, `Accept-Language: en-US,en;q=0.9`, `Content-Type: application/x-www-form-urlencoded`, `content-length: 44`, `Origin: https://192.168.100.177`, `Connection: keep-alive`, `Referer: https://192.168.100.177/dvwa/login.php`, `Cookie: security=low; PHPSESSID=f54d4ac809bf5618dee8f2ec69f46bda`, `Upgrade-Insecure-Requests: 1`, `Sec-Fetch-Dest: document`, and `Sec-Fetch-Mode: navigate`. The response section shows an HTTP/1.1 302 Found status, with headers including `Date: Sat, 02 May 2026 15:01:37 GMT`, `Server: Apache/2.2.8 (Ubuntu) DAV/2`, `X-Powered-By: PHP/5.2.4-2ubuntu5.10`, `Expires: Thu, 19 Nov 1981 08:52:00 GMT`, `Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0`, `Pragma: no-cache`, `Location: login.php`, `Content-Length: 0`, `Keep-Alive: timeout=15, max=98`, `Connection: Keep-Alive`, and `Content-Type: text/html`.

```
POST http://192.168.100.177/dvwa/login.php HTTP/1.1
host: 192.168.100.177
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Content-Type: application/x-www-form-urlencoded
content-length: 44
Origin: https://192.168.100.177
Connection: keep-alive
Referer: https://192.168.100.177/dvwa/login.php
Cookie: security=low; PHPSESSID=f54d4ac809bf5618dee8f2ec69f46bda
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate

HTTP/1.1 302 Found
Date: Sat, 02 May 2026 15:01:37 GMT
Server: Apache/2.2.8 (Ubuntu) DAV/2
X-Powered-By: PHP/5.2.4-2ubuntu5.10
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate, post-check=0, pre-check=0
Pragma: no-cache
Location: login.php
Content-Length: 0
Keep-Alive: timeout=15, max=98
Connection: Keep-Alive
Content-Type: text/html
```

The credentials werenot echoed back by the server

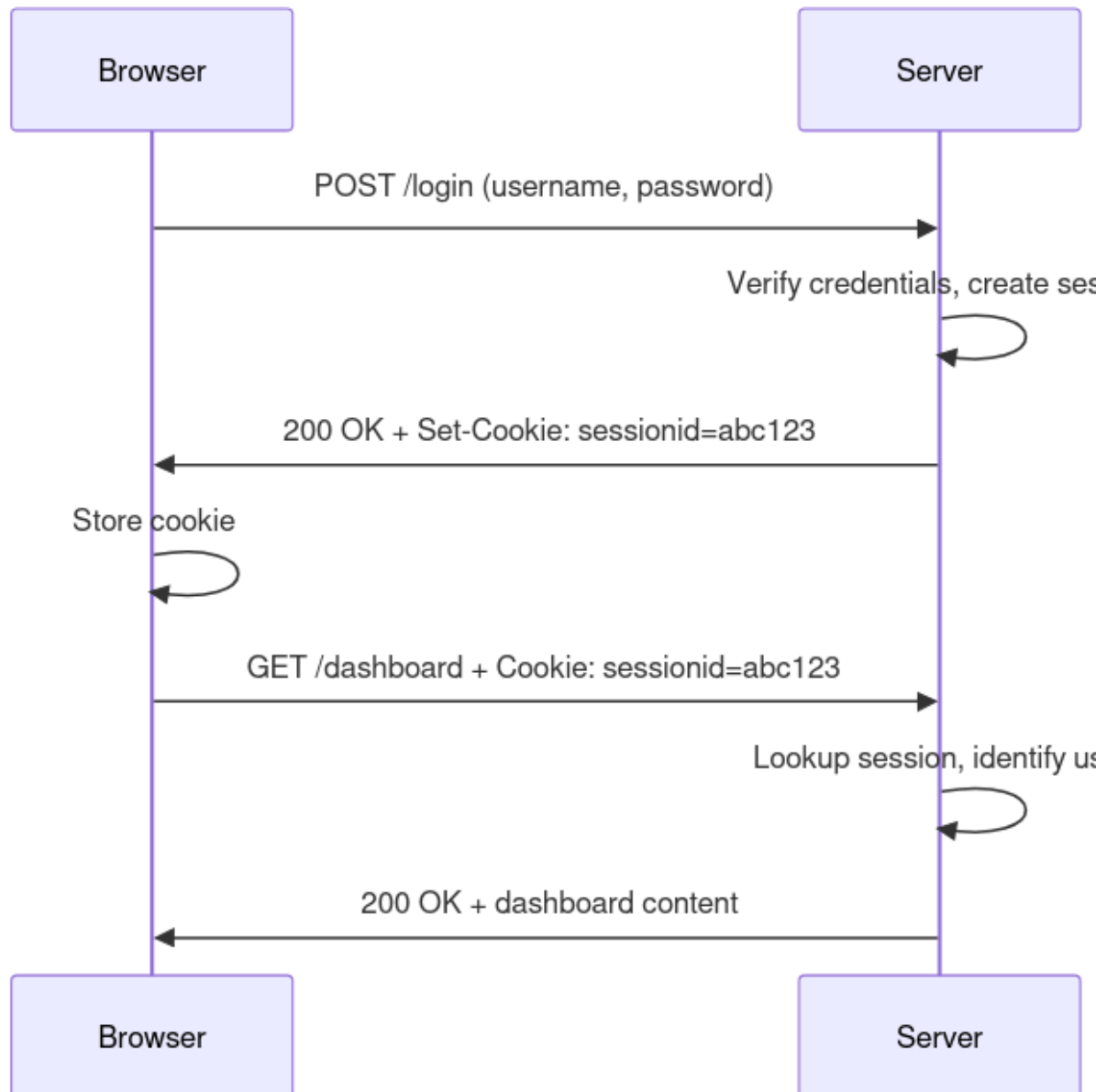
Now for a different directory



The screenshot shows a web browser's developer tools interface. The top bar has tabs for 'Quick Start', 'Request', 'Response', 'Requester', 'Break', and a plus sign. The 'Request' tab is selected. Below the tabs, there are two dropdown menus: 'Header: Text' and 'Body: Text'. The main area displays the details of an HTTP GET request to `http://192.168.100.177/dvwa/vulnerabilities/brute/`. The request headers include `host: 192.168.100.177`, `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0`, `Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8`, `Accept-Language: en-US,en;q=0.9`, `Connection: keep-alive`, `Referer: https://192.168.100.177/dvwa/index.php`, `Cookie: security=low; PHPSESSID=f54d4ac809bf5618dee8f2ec69f46bda`, `Upgrade-Insecure-Requests: 1`, `Sec-Fetch-Dest: document`, `Sec-Fetch-Mode: navigate`, `Sec-Fetch-Site: same-origin`, `Sec-Fetch-User: ?1`, and `Priority: u=0, i`.

```
GET http://192.168.100.177/dvwa/vulnerabilities/brute/ HTTP/1.1
host: 192.168.100.177
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:149.0) Gecko/20100101 Firefox/149.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.9
Connection: keep-alive
Referer: https://192.168.100.177/dvwa/index.php
Cookie: security=low; PHPSESSID=f54d4ac809bf5618dee8f2ec69f46bda
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: same-origin
Sec-Fetch-User: ?1
Priority: u=0, i
```


Theory:



Are cookies and sessions important for modern web:

Yes, HTTP is stateless by design. Without cookies/sessions every request would be anonymous. They are the only mechanism that gives web apps memory across requests. Every modern auth system depends on them.

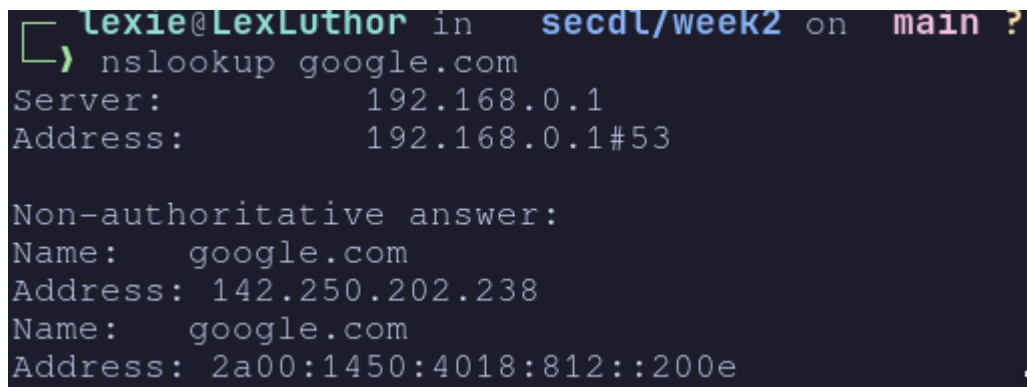
AuthN (Authentication): The process of verifying identity proving you are who you claim to be. Example: submitting username and password.

AuthZ (Authorization): The process of verifying permissions determining what an authenticated identity is allowed to do. Example: checking if you can access `/admin`.

One difference: AuthN happens once at login. AuthZ happens on every single request.

Why cookies/sessions are used: To simulate statefulness on top of a stateless protocol. The server issues a session ID after AuthN, stored in a cookie, which the browser sends automatically on every subsequent request so the server can identify you.

Task 5:

A terminal window with a dark background. The prompt is 'lexie@LexLuthor in secdl/week2 on main ?'. The command 'nslookup google.com' has been entered. The output shows two DNS records for google.com: one with IP 142.250.202.238 and another with IPv6 address 2a00:1450:4018:812::200e.

```
lexie@LexLuthor in secdl/week2 on main ?
> nslookup google.com
Server:      192.168.0.1
Address:     192.168.0.1#53

Non-authoritative answer:
Name:   google.com
Address: 142.250.202.238
Name:   google.com
Address: 2a00:1450:4018:812::200e
```

What is DNS: A distributed system that translates human-readable domain names into machine-readable IP addresses. It is the internet's phonebook.

What is a DNS request: A query sent by your machine asking "what IP address does this domain name map to?"

What is a DNS response: The answer returned by a nameserver containing the IP address (or an error if not found).

Domain Vs IP

Domain name

IP address

Domain Vs IP

Human-readable	Machine-readable
----------------	------------------

google.com	142.250.180.46
------------	----------------

Easy to remember	Required for routing
------------------	----------------------

Can change mapping	Actual network address
--------------------	------------------------

Why DNS is important: Without it you'd need to memorize IP addresses for every site. It also allows a domain to point to different IPs over time (load balancing, CDN, migration) without users noticing.